PowerTrader: Fitting & Trading Algorithm

A technical reference derived entirely from source code analysis

1. System Overview

PowerTrader is a pattern-matching trading system. It has three core components that run as separate processes and communicate through the filesystem:

Trainer (*pt_trainer.py*) — Fetches historical candle data from KuCoin for each of 7 timeframes. Walks through price history candle-by-candle, building a database of "memories" — observed price patterns paired with what happened next. Adjusts memory weights based on prediction accuracy. Adapts a matching threshold to control selectivity.

Thinker (*pt_thinker.py*) — The real-time inference engine ("neural runner"). Continuously cycles through all 7 timeframes per coin. For each, it compares the current candle's shape to stored memories. When matches are found, it averages their weighted predictions to estimate where price will go. Produces a LONG, SHORT, or WITHIN signal per timeframe. Counts total LONGs and SHORTs across all 7 to produce an aggregate signal strength (0–7).

Trader (*pt_trader.py*) — Reads the signal files written by the Thinker. Uses them as a gate: only enters new positions when the long signal is strong enough and the short signal is zero. Manages open positions with configurable DCA (dollar-cost averaging) at hard percentage loss levels or neural price levels, and exits via a trailing profit margin.

The key insight: no machine learning model is trained in the traditional sense. Instead, the system stores raw historical patterns and retrieves them by similarity — closer to a nearest-neighbor lookup than a neural network. The "neural" name is a branding choice, not an architecture description.

2. Training: Building the Memory Database

2.1 Data Acquisition

The trainer fetches historical OHLCV candle data from KuCoin's public API for the coin being trained (e.g., BTC-USDT). It requests up to 100,000 candles in paginated batches of ~1,500, working backwards in time. This is done independently for each of the 7 timeframes:

```
1hour · 2hour · 4hour · 8hour · 12hour · 1day · 1week
```

Each candle provides: open, close, high, low, volume. The trainer computes percentage change features for each:

```
close_pct = 100 × (close - open) / open
high_pct = 100 × (high - open) / open
low_pct = 100 × (low - open) / open
```

This normalization is critical — by converting absolute prices to percentage changes, the same memory can match patterns whether BTC is at $30k or $90k.

2.2 Memory Structure

Each memory is a text record stored in `memories_{tf}.txt`, separated by ~. The format is:

Fig 1. Memory record format: pattern features separated from prediction targets by {} delimiters

The **pattern** portion contains the close percentage change(s) of the observed candle(s) — this is what gets matched against. The last element is the "next candle" close % change that was actually observed. The **high** and **low** fields store what the next candle's high and low percentage changes were — these are the prediction targets.

With `number_of_candles = [2]` (the current setting), each pattern consists of a single candle's close % change plus the next candle's close % change. The system uses these as a 1-candle lookback context.

2.3 The Training Loop

The trainer walks through historical price data candle-by-candle, from past to present. At each step it:

1. Extracts the current candle's close % change as the **query pattern**.
2. Compares this pattern against every stored memory using **percentage difference**.
3. If any memory's pattern is within the **adaptive threshold**, it's a "match".

4. **If matches exist**: the system computes weighted-average predicted moves, then checks how the *actual* next candle compared. It adjusts the weights of matching memories up or down based on accuracy.

5. **If no matches exist**: the current candle + what actually happened next is stored as a **new memory** with weight 1.0.

6. Advance to the next candle and repeat.

The training processes all 7 timeframes sequentially. For each timeframe, it does an initial "warm-up" pass on the 1-hour data (building context), then two passes on the target timeframe — first on the early half, then the full dataset.

2.4 Pattern Matching (Similarity Metric)

The similarity between the current candle and a stored memory is computed as:

```
difference = |current - memory| / ((current + memory) / 2) × 100
```

This is a symmetric percentage difference (similar to MAPE). A difference of 0 means identical candle shapes; higher values mean less similarity. The candle is a "match" if the difference is ≤ the adaptive threshold for that timeframe.

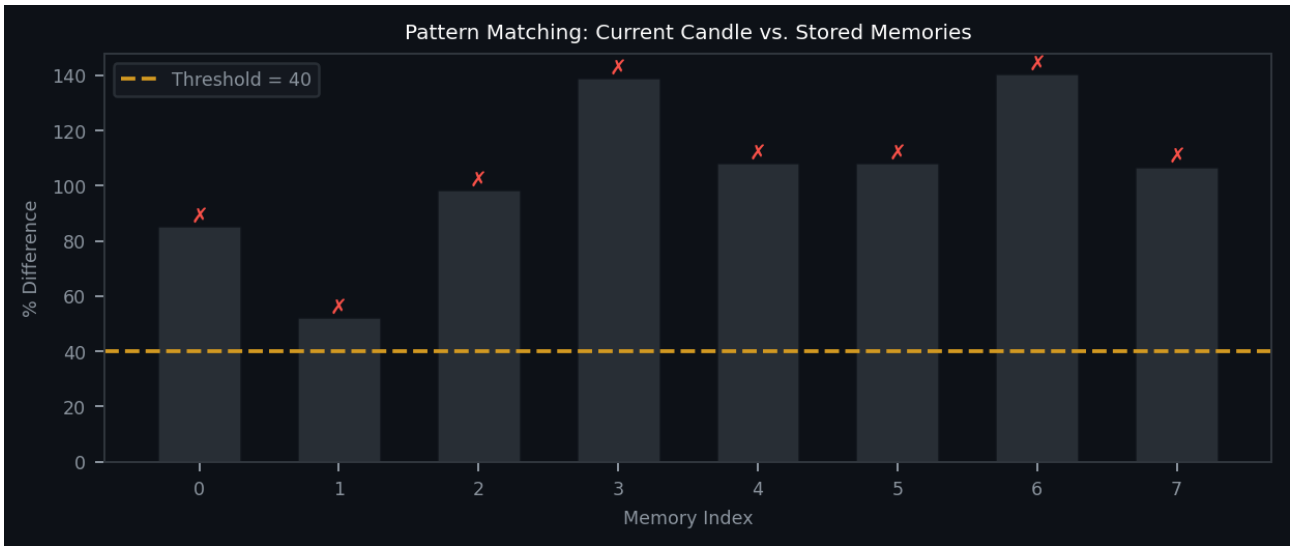

Fig 2. Pattern matching: only memories within the threshold (green) contribute to the prediction

2.5 Adaptive Threshold

The threshold is not fixed — it self-adjusts during training to maintain a useful number of matches per pattern. The logic:

- If more than 20 memories match → threshold **decreases** (become more selective)
- If 20 or fewer match → threshold **increases** (become less selective)
- Step size: ±0.01 when threshold ≥ 0.1, ±0.001 below 0.1
- Bounded between 0.0 and 100.0

This creates a self-regulating system: coins with many similar candle shapes (low volatility) get a tight threshold, while coins with diverse patterns get a wider one.

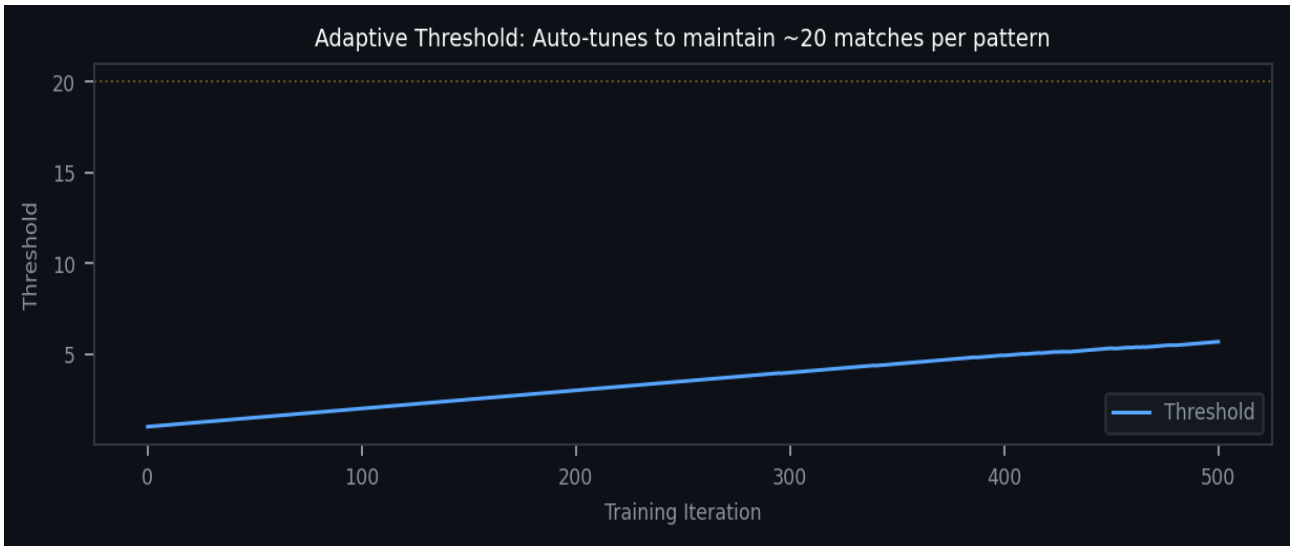

Fig 3. Threshold converges to a level that maintains a moderate number of matches

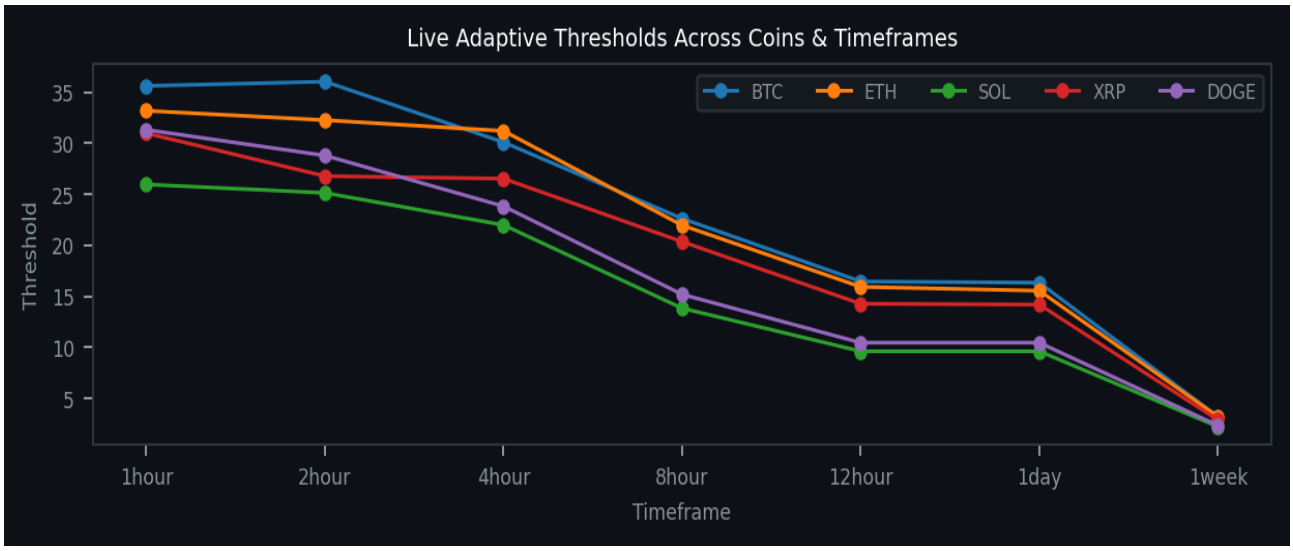

Fig 4. Live thresholds from your trained models — shorter timeframes have wider thresholds (more pattern diversity)

2.6 Weight Adjustment

When memories match the current candle, the trainer checks what actually happened vs. what the memory predicted. Three independent weights are adjusted:

Close weight — stored in `memory_weights_{tf}.txt`
High weight — stored in `memory_weights_high_{tf}.txt`
Low weight — stored in `memory_weights_low_{tf}.txt`

For each matching memory, the adjustment rule is:

- If actual move > predicted move × 1.1 (underpredicted by 10%+): weight += 0.25
- If actual move < predicted move × 0.9 (overpredicted by 10%+): weight –= 0.25
- Otherwise (prediction within 10% of actual): weight unchanged

Close weights are clamped to [–2.0, 2.0]. High/low weights are clamped to [0.0, 2.0]. All new memories start with weight 1.0.

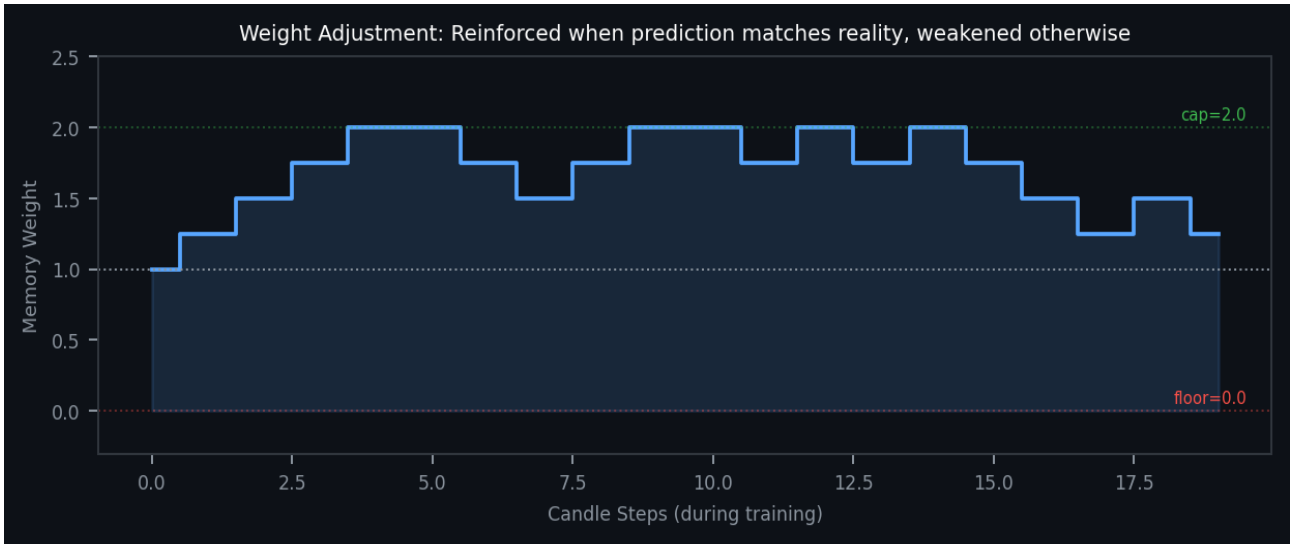

Fig 5. Weight evolution: accurate memories are reinforced, poor predictions are dampened

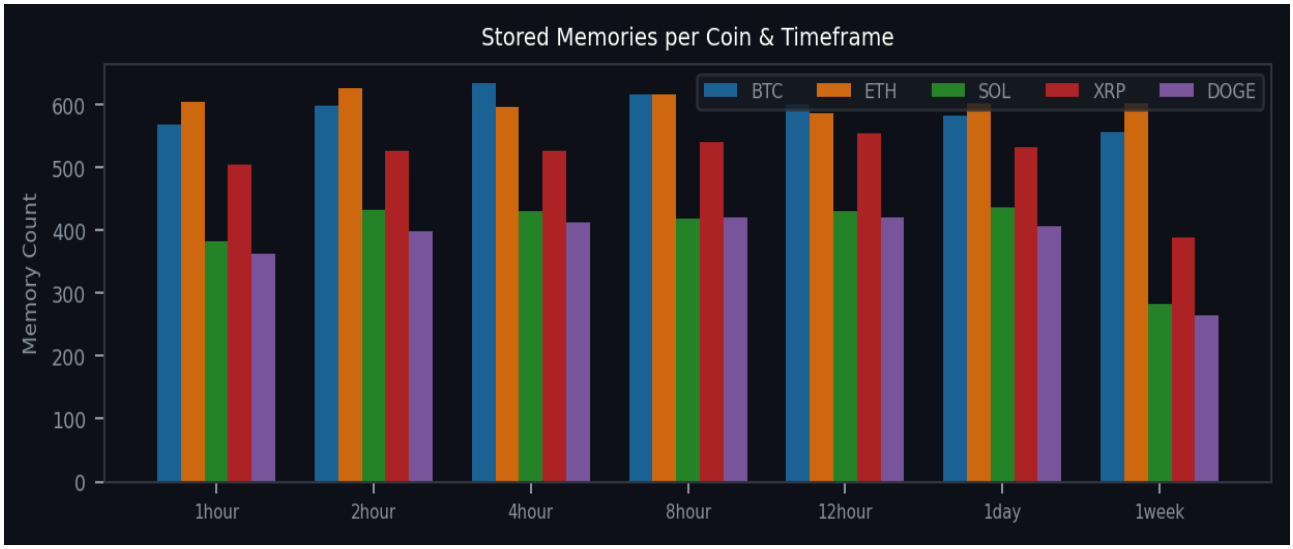

Fig 6. Actual memory counts across your trained coins — typically 500–650 per timeframe

3. Inference: From Candles to Trading Signals

3.1 The Thinker Loop

The Thinker (`pt_thinker.py`) runs continuously, cycling through each configured coin. For each coin, it steps through the 7 timeframes one at a time. After completing a full sweep of all 7, it produces output signals.

For each timeframe step, the Thinker:

1. Fetches the most recent candle from KuCoin.
2. Computes the candle's close % change: `100 × (close – open) / open`.
3. Loads the stored memories and weights for this timeframe.
4. Reads the adaptive threshold from `neural_perfect_threshold_{tf}.txt`.
5. Compares the current candle against each stored memory using the same percentage-difference metric as training.
6. Collects all matching memories (difference ≤ threshold) and their associated weighted predictions.

3.2 Computing Predicted Moves

When matching memories are found, the Thinker computes three weighted averages:

```
final_moves = Σ(pattern_close_pct × close_weight) / n_matches
high_final_moves = Σ(high_pct × high_weight) / n_matches
low_final_moves = Σ(low_pct × low_weight) / n_matches
```

Zero-weight memories are excluded from their respective averages. The predicted future prices are:

```
predicted_high = current_close + (current_close × high_final_moves)
predicted_low = current_close + (current_close × low_final_moves)
```

3.3 Price Bounds & Signal Classification

The predicted prices are converted to **bounds** with a 0.5% distance margin:

```
low_bound = predicted_low – (predicted_low × 0.005)
high_bound = predicted_high + (predicted_high × 0.005)
```

The bounds are then sorted across all 7 timeframes and spread apart to ensure a minimum 0.25% gap between adjacent levels. This creates a ladder of N1–N7 price levels for both longs and shorts. The signal for each timeframe is:

• **LONG** — current price is below the low bound (price is cheap relative to prediction)
• **SHORT** — current price is above the high bound (price is expensive relative to prediction)
• **WITHIN** — current price is between the bounds (no strong directional signal)
• **INACTIVE** — no matching memories were found for this timeframe

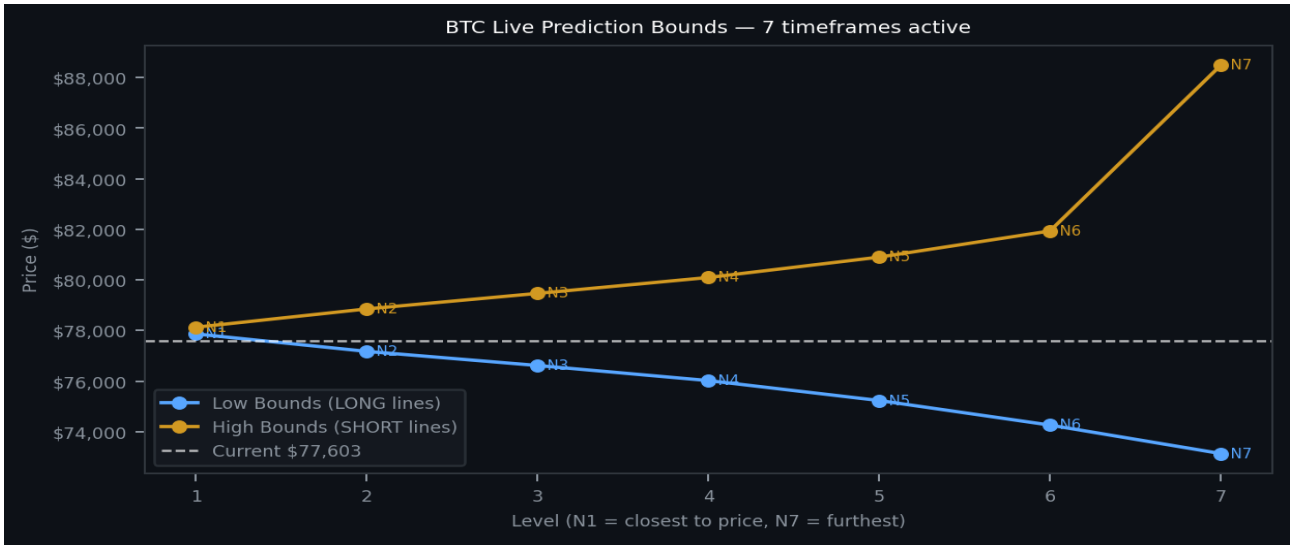

Fig 7. Live BTC prediction bounds — blue (LONG) lines below price, orange (SHORT) lines above

3.4 Signal Aggregation

After completing a full sweep of all 7 timeframes, the Thinker counts the total number of LONG and SHORT signals and writes them to files:

```
long_dca_signal.txt → count of timeframes signaling LONG (0–7)
short_dca_signal.txt → count of timeframes signaling SHORT (0–7)
```

A value of 7 means all timeframes agree — the strongest possible signal. A value of 0 means no timeframes are signaling in that direction.

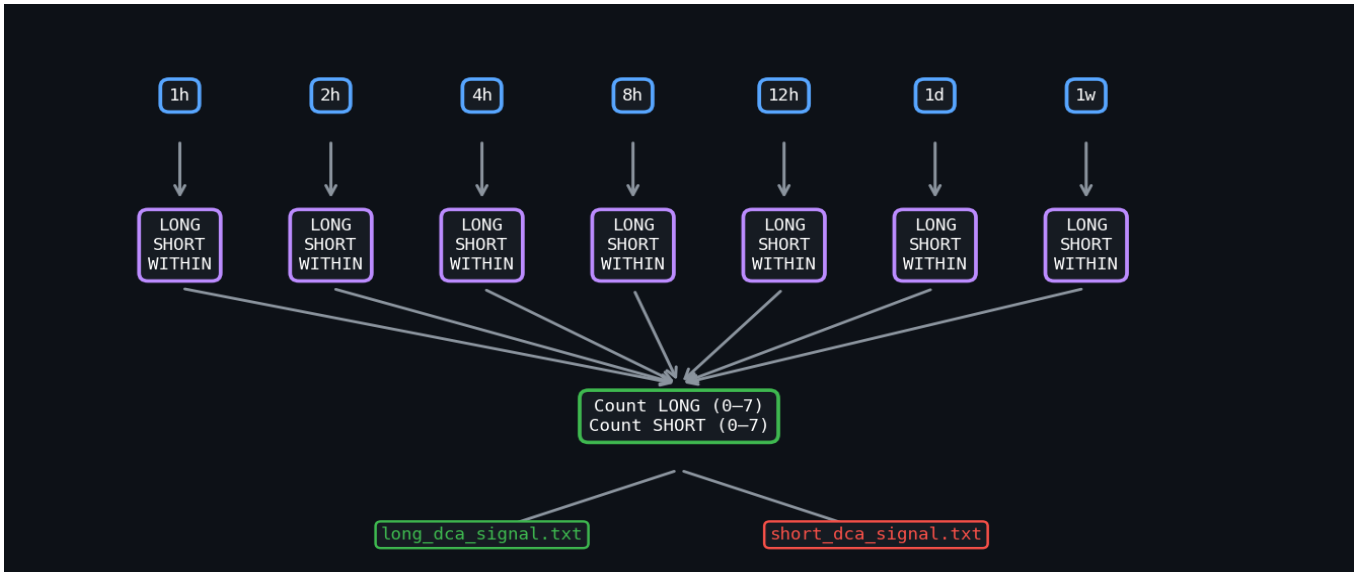

Fig 8. Signal pipeline: 7 independent timeframe assessments aggregate into a single strength score

Fig 9. Distribution of stored prediction targets in BTC 4h memories — the raw material for weighted averaging

4. Trading: From Signals to Positions

4.1 Entry Gate

The Trader reads the signal files and applies a strict gate before entering any new position:

```
ENTER if: long_signal ≥ TRADE_START_LEVEL AND short_signal == 0
```

TRADE_START_LEVEL is configurable (default 4, range 1–7). A setting of 4 means at least 4 out of 7 timeframes must be signaling LONG, and *zero* must be signaling SHORT. This is deliberately conservative — it only trades when there is broad consensus across timeframes and no contradicting signals.

4.2 Position Sizing

Each new position is sized as a percentage of total account value:

```
allocation = total_account_value × (start_allocation_pct / 100)
```

The default `start_allocation_pct` is 0.5%, meaning a $10,000 account would start each position at $50. This conservative sizing allows the DCA mechanism to scale in with larger amounts if the price drops.

4.3 Dollar-Cost Averaging (DCA)

If an open position goes into loss, the Trader may add to it ("DCA") at predefined trigger levels. Two independent triggers exist — whichever fires first activates the DCA:

Hard % triggers — fixed loss percentages from cost basis: [-5%, -10%, -20%, -30%, -40%, -50%, ...]
Neural level triggers — when the long signal reaches a higher neural level AND price has actually reached the corresponding neural price line. For `trade_start_level=4`, the neural DCA stages are N5, N6, N7 (the levels below the start level).

Each DCA buy is sized at: `current_position_value × dca_multiplier` (default 2×). This means each DCA buy is larger than the previous, heavily weighting the lower prices — a classic Martingale-influenced approach that rapidly lowers the average entry.

Rate limit: maximum `max_dca_buys_per_24h` DCA buys per coin per rolling 24-hour window.

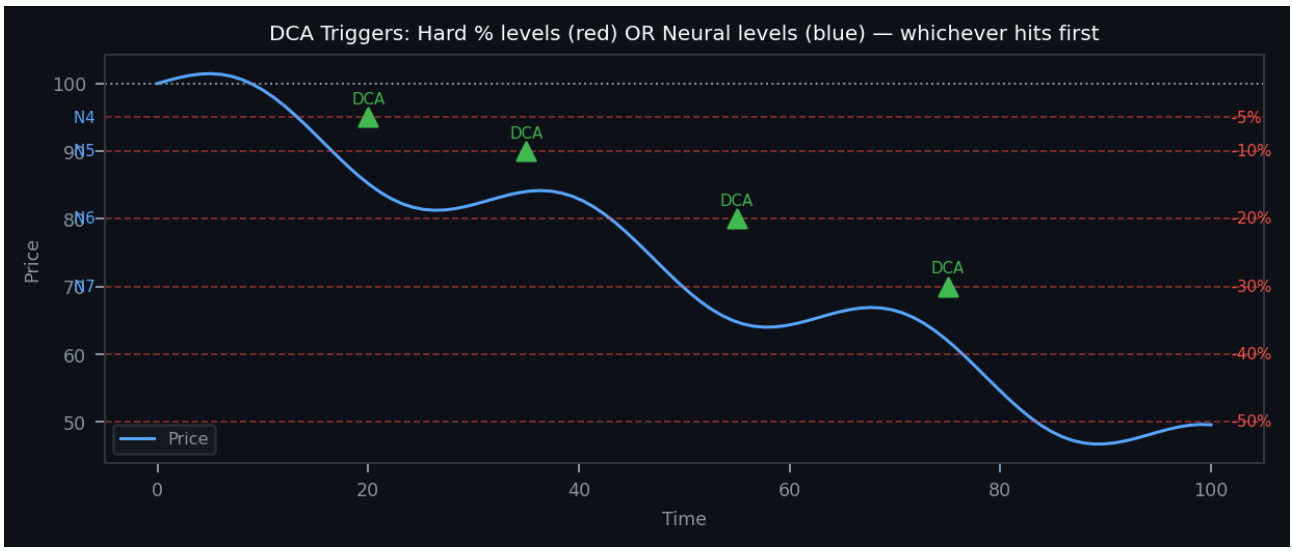

Fig 10. DCA trigger levels: hard % (red) and neural (blue) — first to fire triggers the buy

4.4 Trailing Profit Margin (Exit)

Exits are controlled by a trailing profit margin mechanism — there is no fixed take-profit target. The system:

1. Computes a **PM start line** above the average entry: cost_basis × (1 + pm_start_pct%). Different thresholds for positions with DCA vs. without (default: +3% without DCA, +3% with DCA).

2. When price rises above the PM start line, **trailing activates**. The system records the peak price.

3. The trailing line = peak × (1 – trailing_gap_pct%). Default gap is 0.1%, so the line stays very close to the peak.

4. As price continues up, the peak and trailing line ratchet up with it. The line can never move down.

5. **SELL triggers** when price drops from above the trailing line to below it. This means the system captures most of an upward move, only giving back the trailing gap amount.

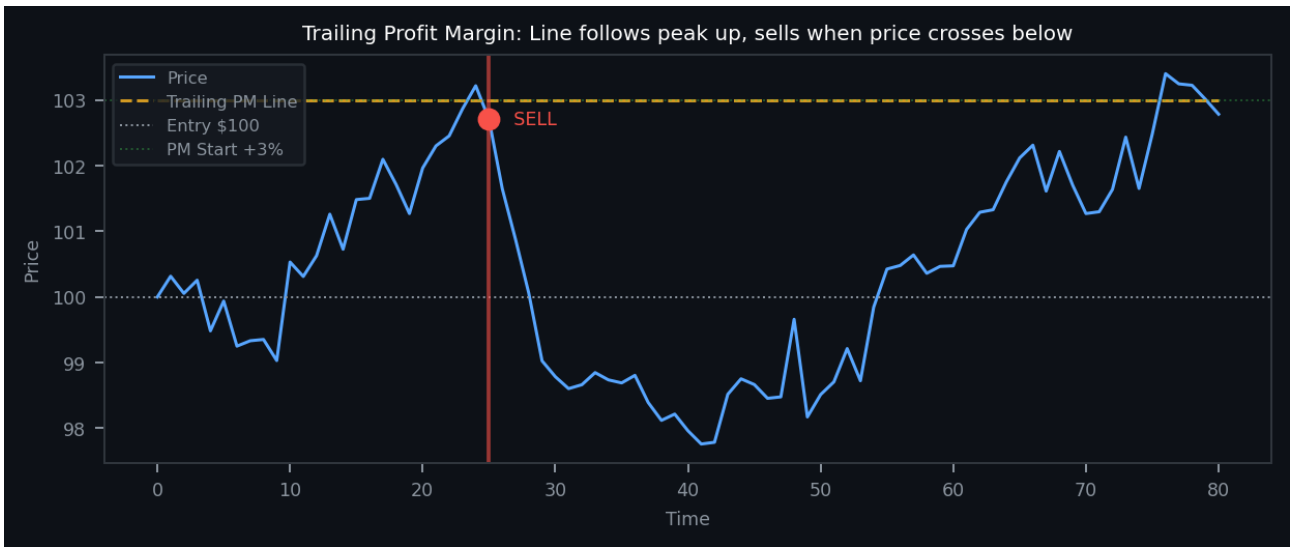

Fig 11. Trailing PM: line follows price up, sell triggers on the first cross below

5. End-to-End Example

Here is a complete walkthrough of how a trade might unfold for BTC:

Step	Action	Detail
1	Thinker scans BTC	Current 1h candle: open=$77,500, close=$77,200 → pct_change = –0.39%. Compares against 568 stored 1h memor
2	Repeat × 7 TFs	Thinker completes all 7 timeframes. 5 timeframes signal LONG, 0 signal SHORT. Writes long_dca_signal.txt = 5, sho
3	Trader reads signals	With trade_start_level = 4: long (5) ≥ 4 ✓, short (0) == 0 ✓. Entry gate PASSES.
4	Initial buy	Account = $10,000. allocation = $10,000 × 0.5% = $50. Buys $50 of BTC at $77,200.
5	Price drops –6%	BTC falls to $72,568. Hard DCA trigger at –5% fires. Current position value ≈ $47. DCA buy = $47 × 2 = $94 at $72,5
6	Price recovers	BTC rises to $76,300. Position PnL = +3.2%. PM start line (with DCA) = $73,959 × 1.03 = $76,178. Price > PM line –
7	Trailing ratchets up	BTC reaches $77,100. Peak = $77,100. Trail line = $77,100 × (1 – 0.001) = $77,023.
8	Exit	BTC pulls back to $76,950 (below $77,023). TRAIL_SELL fires. Position closed at ~+4.0% profit.

6. Key Properties of the Algorithm

Non-parametric — No model weights are learned in the ML sense. The system stores raw patterns and retrieves them by similarity. This is conceptually similar to k-nearest-neighbors with adaptive k (controlled by the threshold).

Self-regulating threshold — The match threshold adapts during training so the system always works with a manageable number of matches, regardless of the coin's volatility profile.

Multi-timeframe consensus — The entry gate requires agreement across multiple timeframes, acting as a noise filter. Short-term noise on a single timeframe won't trigger trades; only broad alignment does.

Asymmetric risk management — Small initial positions with aggressive DCA on drops means the system sizes up only at lower prices. The trailing exit captures upside without a fixed target.

Staleness protection — Coins that haven't been trained within 14 days have their signals zeroed out by the Thinker, preventing stale memories from triggering trades.

Weight decay — Memories that consistently mispredict have their weights reduced toward zero, effectively silencing them without deleting them from the database. Accurate memories accumulate weight up to the cap of 2.0, giving them more influence on predictions.